System Architecture

The system is designed as a web-based pipeline that processes Power Platform solution files and generates structured and human-readable documentation.

A user uploads a solution ZIP through a web interface, which is sent to the backend via a single `/generate` endpoint. The backend orchestrates parsing, data structuring, and output generation using both deterministic logic and a RAG-based generation layer.

The architecture separates concerns between:

- user interaction (frontend)
- processing and orchestration (backend)
- structured data extraction (parser)
- output generation (deterministic and RAG)

This separation ensures the system remains modular, scalable, and easy to extend.

Technology Stack

The system is implemented using the following technologies:

- Frontend: Web UI (React-based interface)
- Backend: ASP.NET Core (C#)
- Parser: Custom C# solution parser
- Intermediate Data: JSON chunk files
- Deterministic Generation: Rule-based backend logic
- RAG Pipeline: OpenAI vector store + LLM generation
- Output Formats: Markdown, Mermaid diagrams, downloadable files

This combination allows the system to produce both structured and human-readable outputs efficiently.

Deployment Context

The system is currently designed to run in a controlled environment, with the frontend communicating directly with the backend API.

The architecture supports containerised deployment using Docker where the frontend and backend can run as separate services or a combined service using docker_compose.yml. This allows consistent environments across development and deployment and simplifies scaling and distribution.

In a production setting, the backend is responsible for parsing, generation, and temporary file storage, while external LLM services are accessed via the user's own Azure workspace. In addition, an admin user can adjust configuration settings to update both backend and frontend behaviour.

Main Components

1. Web UI

The frontend allows the user to:

- upload a Power Platform solution ZIP
- trigger analysis
- preview generated outputs
- download generated files

It does not perform processing itself; it only sends requests and displays results.

2. /generate Endpoint

The frontend communicates with the backend through a single endpoint: /generate.

This endpoint triggers the full processing pipeline. This design keeps the frontend simple and stable while allowing internal backend logic to evolve without changing the API.

Request:

- Method: POST
- Content-Type: multipart/form-data
- Input: Power Platform solution ZIP file

Response:

- Generated outputs returned as downloadable files (e.g. Markdown, diagrams)
- May include preview-ready content for frontend display

Error Handling:

- Invalid file → validation error
- Parsing failure → handled without crashing pipeline
- Generation failure → partial results may still be returned

3. Backend Orchestrator

The orchestrator manages the full workflow:

- receives and validates the uploaded file
- calls the parser
- routes structured data into generation layers
- returns generated outputs

It acts as the central control point for the pipeline.

4. Solution Parser

The parser extracts structured information from the uploaded solution.

Responsibilities:

- Canvas App extraction
- workflow parsing
- environment variable extraction
- model-driven app detection
- relationship mapping

The parser converts the raw ZIP into structured intermediate data.

5. Structured JSON Chunks

Parsed data is stored as structured JSON chunks, which act as the interface between parsing and generation.

Typical chunk categories:

- overview
- workflows
- detailed workflows
- canvas apps

- relationships
- environment variables
- model-driven apps

This separation allows the system to reuse parsed data across multiple output types.

6. Deterministic Generation Layer

This layer generates outputs that must remain structurally accurate and consistent.

Used for:

- ER diagrams
- system diagrams
- workflow mappings
- relationship graphs

These outputs rely on explicit logic rather than model inference, ensuring correctness and repeatability.

7. RAG Pipeline

The RAG (Retrieval-Augmented Generation) layer produces natural-language outputs.

It works as follows:

- structured JSON chunks are embedded and stored in a vector store
- relevant chunks are retrieved based on the generation task
- an LLM uses retrieved context to generate responses

Used for:

- overview documentation
- FAQ generation
- human-readable explanations

RAG is used here because these outputs require interpretation and summarisation rather than strict structural accuracy.

8. Output Handling

Generated outputs are returned to the frontend as a response package.

The frontend:

- holds output package in browser memory
- previews outputs directly
- allows users to download files

This avoids additional API calls and keeps the interaction responsive.

Authentication and Authorisation

The current system assumes a controlled environment and does not implement user authentication. This is a known limitation. In a production system, authentication would be required to control access to file uploads and generated outputs.

Non-Functional Considerations

Scalability:

The system can support concurrent processing by introducing background workers or queue-based execution.

Performance:

Processing time depends on solution size. Chunk-based processing helps manage large inputs efficiently.

Reliability:

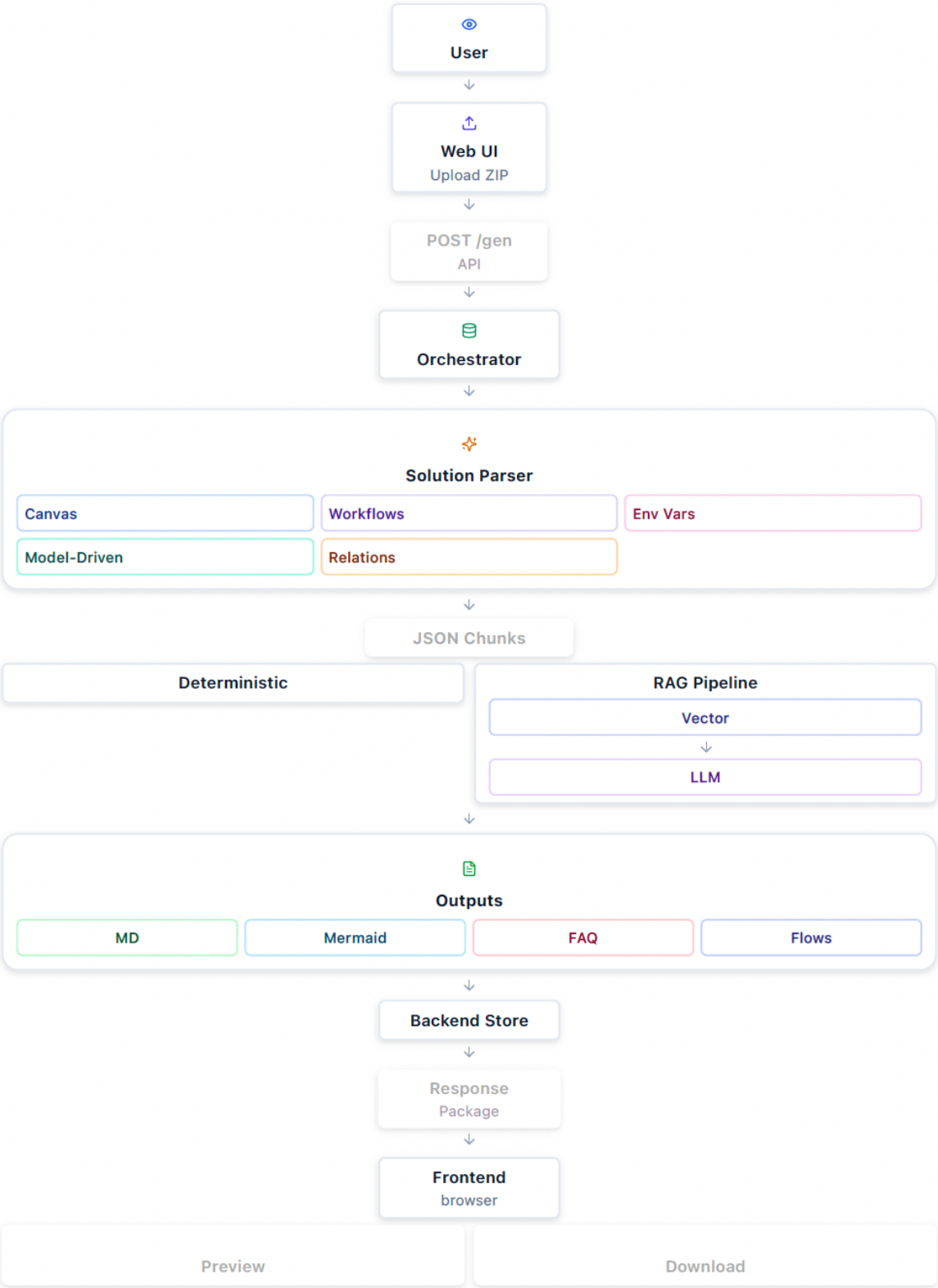
Validation and error handling ensure invalid files or failures do not break the system.

Security:

Uploaded files and related data are retained only for a limited time and are automatically deleted after the configured TTL (Time-To-Live) period, ensuring that client data is not stored beyond this window.

Extensibility:

The modular pipeline (parser → chunks → generation layers) allows new outputs or analysis features to be added easily.

Solution Documentation Pipeline

Automated documentation generation workflow